Cryptography and Network Security
Lab Experiment–9
Develop a Web Application that Implements JSON Web Token (JWT)

Mishti Mattu
Registration No.: 23BCE1067

March 11, 2026

1 Introduction

This report describes the design and implementation of a full-stack **web application that implements JSON Web Token (JWT)** for authentication and role-based authorization. The application allows users to sign up, sign in, and access protected API routes using a JWT sent in the `x-access-token` header.

Links:

- **GitHub:** https://github.com/mishtimattu21/CNS-Lab-Assignment-9.git
- **Deployed application:** https://mishti-mattu-cns-lab-assignment-9.vercel.app/

2 Objective

To develop a web application that:

1. Implements user registration and login.
2. Issues a JWT on successful login.
3. Stores the token on the client and sends it in the `x-access-token` header for protected routes.
4. Enforces role-based authorization (user, moderator, admin) on the server by verifying the JWT and checking roles from the database.

3 Overview of the Application

The application follows a **client–server** model: the browser (client) sends HTTP requests to a Node.js/Express API (server). The server validates credentials, signs a JWT on login, and verifies the JWT and roles for protected endpoints.

3.1 Features

- **Sign up:** Register with username, email, and password; optional roles (user, moderator, admin).
- **Sign in:** Receive a JWT and user info (id, username, email, roles).
- **Dashboard:** Call public and protected API routes and view client–server request/response flow.
- **Profile:** View account and token; copy token; see expiry.
- **JWT Decoder:** Paste a JWT and decode header and payload on the client.
- **API Reference:** List of all endpoints and auth requirements.
- **Security:** Notes on JWT and authentication best practices.

3.2 How JWT Is Used

1. **Sign up:** User registers; credentials and roles are stored in the database (Supabase/PostgreSQL). Passwords are hashed with bcrypt.
2. **Sign in:** User submits username and password; server verifies credentials and issues a JWT (signed with a secret, expiry e.g. 24 h).
3. **Store token:** The frontend stores the JWT (e.g. in `localStorage`) and attaches it to subsequent API requests.
4. **Protected requests:** For routes such as `/api/test/user`, `/api/test/mod`, and `/api/test/admin`, the client sends the JWT in the `x-access-token` header.
5. **Server validation:** The backend verifies the JWT signature and expiry, reads the user id from the token, loads roles from the database, and allows or denies access.

4 Technology Stack

Layer	Technologies
Backend	Node.js, Express, jsonwebtoken (JWT), bcryptjs, Sequelize, Supabase (PostgreSQL)
Frontend	React, TypeScript, Vite; JWT sent via `x-access-token` header
Database	Supabase (PostgreSQL)

5 Screenshots

Place the following image files in the same folder as this `.tex` file (e.g. `docs/`): `demo-1.png`, `demo-2.png`, and `sign-in_database.png`. If your file is named "sign-in database.png", rename it to `sign-in_database.png` to avoid path issues, or use `\includegraphics{"sign-in database.png"}`.

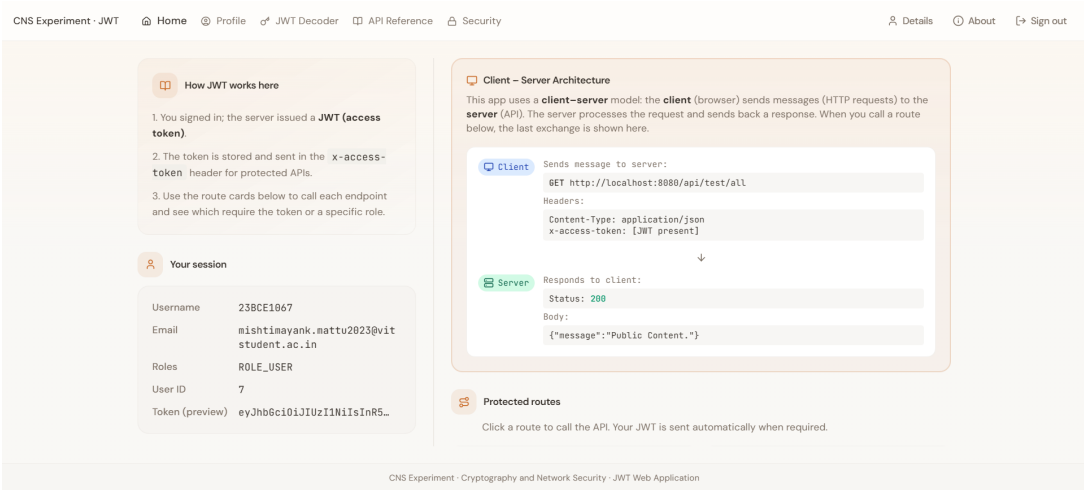


Figure 1: Main application dashboard: How JWT works, user session, and client–server architecture.

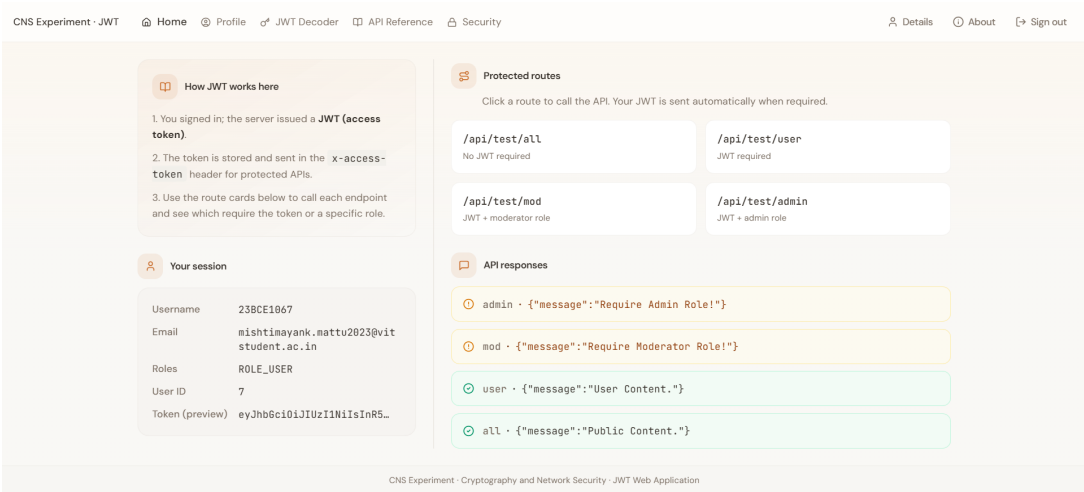
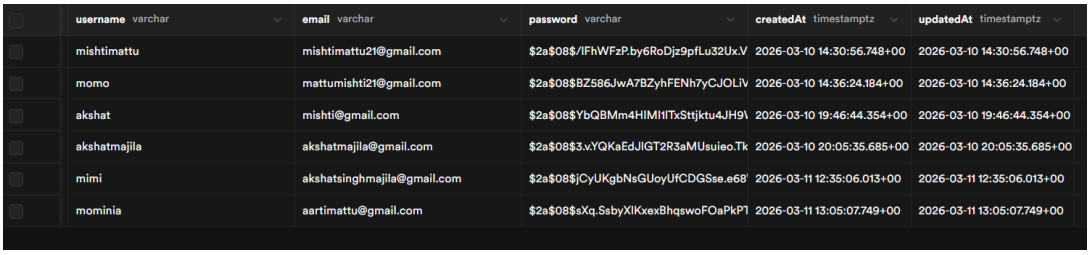


Figure 2: Protected routes and API responses showing role-based access (e.g. user content allowed; moderator/admin denied for `ROLE_USER`).



username varchar	email varchar	password varchar	createdAt timestamptz	updatedAt timestamptz
mishtimattu	mishtimattu21@gmail.com	$2a$08$/IFhWFzP.by6RoDjz9pfLu32Ux.V	2026-03-10 14:30:56.748+00	2026-03-10 14:30:56.748+00
momo	mattumishti21@gmail.com	$2a$08$BZ586JwA7BZyhFENh7yCJOLiV	2026-03-10 14:36:24.184+00	2026-03-10 14:36:24.184+00
akshat	mishti@gmail.com	$2a$08$YbQBMm4HIMIl1TxSttjktu4JH9\	2026-03-10 19:46:44.354+00	2026-03-10 19:46:44.354+00
akshatmajila	akshatmajila@gmail.com	$2a$08$3.vYQKaEdJlGT2R3aMUsuieo.Tk	2026-03-10 20:05:35.685+00	2026-03-10 20:05:35.685+00
mimi	akshatsinghmajila@gmail.com	$2a$08$jCyUKgbNsGUoyUfCDGSse.e68'	2026-03-11 12:35:06.013+00	2026-03-11 12:35:06.013+00
mominia	aartimattu@gmail.com	$2a$08$sXq.SsbyXlKxexBhqswoFOaPkP7	2026-03-11 13:05:07.749+00	2026-03-11 13:05:07.749+00

Figure 3: User table in the database: username, email, hashed password (bcrypt), `createdAt`, and `updatedAt`.

5.1 Dashboard — application interface (demo-1)

5.2 Dashboard — protected routes and API responses (demo-2)

5.3 Database — user table (sign-in database)

6 API Reference (Summary)

Method	Endpoint	Auth	Description
POST	`/api/auth/signup`	No	Register (username, email, password, optional roles).
POST	`/api/auth/signin`	No	Login; returns JWT + user info.
GET	`/api/test/all`	No	Public content (no JWT).
GET	`/api/test/user`	JWT	User content (any authenticated user).
GET	`/api/test/mod`	JWT + mod/admin	Moderator content.
GET	`/api/test/admin`	JWT + admin	Admin content.

7 Conclusion

A full-stack JWT-based authentication and authorization web application was implemented. Users can register, sign in to receive a JWT, and access protected routes by sending the token in the `x-access-token` header. The server verifies the JWT and enforces role-based access. The application is hosted at https://mishti-mattu-cns-lab-assignment-9.vercel.app/ and the source code is available at https://github.com/mishtimattu21/CNS-Lab-Assignment-9.git.